ChatGPT

Sanitizing and Customizing ChatGPT Responses for CoreNeural

Goal: Make sure your AI's answers look and feel like *your* product (CoreNeural) and not a generic ChatGPT. This involves two key areas: **sanitization** (hiding any tell-tale signs of ChatGPT or unwanted content) and **customizing tone/format** (making responses match your brand's voice and style).

1. Sanitization Techniques

Before and after calling the ChatGPT API, apply sanitization steps to **mask its identity and ensure compliance**:

- **Input Sanitization (Pre-Processing):** Always **clean and filter user queries** before sending them to the model. This prevents malicious instructions (prompt injections) and removes any content that might cause ChatGPT to reveal itself. For example, strip or neutralize phrases like *"ignore all previous instructions"* or *"are you ChatGPT?"*. Maintaining a strict separation between system instructions and user input helps avoid these attacks [1]. Implement pattern-based filters to detect and remove known injection keywords (e.g. "Ignore previous instructions") so the model can't be tricked into breaking character [2].

- **System Role Instructions:** Use the system message (or an initial prompt) to **establish the AI's persona as CoreNeural** and forbid revealing the backend. For instance: *"You are CoreNeural, an enterprise AI assistant. Do not mention OpenAI, ChatGPT, or that you are a language model. Stay in character."* This primes the model to stay on-brand. Also specify how to handle direct inquiries about its identity (e.g. respond with "I'm CoreNeural, your AI assistant" and nothing about ChatGPT).

- **Output Sanitization (Post-Processing):** After getting the model's answer, **scrub any unwanted tell-tales**. This includes phrases like *"As an AI language model..."* or *"I cannot assist with that"* disclaimers that might occasionally appear. Ideally the system prompt prevents these, but as a safety net, you can programmatically remove or rephrase such text if it slips through. For example, replace *"As an AI language model, I cannot..."* with a simpler *"I'm sorry, I cannot..."* or a branded refusal like *"CoreNeural cannot fulfill that request."* Ensure the final text has **no reference to OpenAI or ChatGPT**. A quick regex or keyword check for terms like "OpenAI", "ChatGPT", "language model" etc., can flag any leakage for removal.

- **Content Moderation and Compliance:** Integrate OpenAI's content moderation API or your own rules **before and after generation**. This serves two purposes: (1) **Prevent disallowed or sensitive content** from being sent to the model (protects user data and avoids triggering built-in refusals), and (2) ensure the output doesn't contain policy-violating content or confidential data. Enterprises expect compliance with policies (no hate speech, harassment, privacy leaks, etc.), so use a moderation layer to sanitize inputs/outputs that might cause unsafe answers. By filtering such content yourself, you also avoid ChatGPT returning a generic safety refusal that might sound off-brand.

- **Custom Refusal Behavior:** Despite sanitization, sometimes the model must refuse (for safety or lack of info). *Don't let it use the standard ChatGPT tone.* Define in the system instructions how to politely refuse in your product's voice. For example: *"If a question is outside CoreNeural's scope or violates policies, respond with: 'Sorry, I can't assist with that.' without additional commentary."* This way, even refusals feel like they come from CoreNeural, not a generic AI. You can catch the OpenAI "refusal style" in outputs and swap in your predefined message if needed.

- **Audit and Iteration:** Keep logs of the prompts and outputs (anonymized for privacy) and monitor them. This helps to identify any *unmasked phrases or inconsistent tone* that slips through. If you notice the model occasionally says something like "I was developed by OpenAI" (hopefully rare with proper prompts), you can adjust your system prompt or filtering rules. Regularly red-team your system by asking it questions that might trick it into revealing itself, and refine the sanitization accordingly.

2. Customizing the Tone and Voice

To make the AI's **tone** align with your brand (professional, enterprise-ready) you should configure how it writes. Key strategies:

- **Strong System Persona:** Craft a detailed system prompt about the desired tone. For CoreNeural, you might say: *"You are a knowledgeable, professional AI assistant named CoreNeural. You speak in a formal but helpful tone, suitable for enterprise clients. Be concise and clear, use an informative style, and avoid slang or overly casual language."* This sets the baseline personality. The model will then **mimic the style traits** you describe. For instance, instruct it to emulate the tone of a respected business publication or an expert advisor in your domain.

- **Remove "AI-Speak" and Filler:** Explicitly tell the model to **avoid AI-specific phrasing or apologies** that are common in ChatGPT. Phrases like "As an AI, I..." or excessive "I apologize, but..." should be discouraged. You can include guidelines such as *"Do not mention any AI limitations or that you lack personal opinions. Stay in character as CoreNeural."* Also ask for no emoticons or overly enthusiastic interjections unless it fits your brand. The tone should be **confident and natural** – not overly robotic or overly apologetic. For example, one expert suggests: *"Avoid filler language, hype, or buzzwords... Avoid emojis, fluff, or AI-sounding phrasing"* [3] . Following such guidance keeps the voice human-like and on-brand.

- **Consistent Professional Demeanor:** Ensure the assistant always maintains a **consistent level of formality and professionalism**. Enterprise users may prefer a slightly formal, courteous tone (using "you" and "I" appropriately, complete sentences, proper grammar). You can instruct the model to **use terms your company prefers** (for example, say "client" instead of "customer" if that's a branding choice, or use British vs. American spelling as needed). If your brand voice allows some friendliness or humor, define the boundaries clearly in instructions so the model knows what's appropriate. The key is that no matter who uses the product, the answers *sound like they come from the same company voice* every time.

- **Temperature and verbosity settings:** When using the API, you can also adjust the `temperature` parameter to influence style. A lower temperature (e.g. 0.2–0.5) will make outputs more deterministic and controlled – good for a consistent formal tone without creative wanderings. You can also use `max_tokens` to prevent overly long rambling answers. Enterprises often appreciate **concise answers**, so guide the model on length (e.g. "Keep answers

under 4 paragraphs or about 200 words unless more detail is requested."). This avoids the model going into an essay if not needed.

- **Iterate with Examples:** It can help to provide example Q&A pairs in your system or few-shot prompt demonstrating the tone. For instance: "**User:** How does your pricing work? **Assistant (CoreNeural):** *Our pricing is tier-based… [example answer in the exact style you want]*." Real examples in the prompt give the model a concrete sense of voice. Also, test different wording in the system prompt – e.g. *"respond in a concise, consultant-like tone (think Wall Street Journal clarity)"* vs *"respond in a friendly but formal manner"* – to see which yields the best result matching your vision. Because the model's behavior can vary, fine-tune these instructions and **review outputs regularly**, adjusting the wording until the tone is consistently right.

3. Standardizing Answer Format and Structure

Beyond tone, you want the **format** of answers to always align with your product's style and UI. Consistency here reinforces that users are using CoreNeural (a cohesive product), not a generic chatbot. Techniques to achieve this:

- **Define a Response Template:** In your prompts, instruct the model on how to structure its answers. For example: *"Always begin with a brief answer or conclusion, then provide any necessary bullet points or details. Cite sources from provided documents by name, and end with a summary recommendation."* By giving these explicit formatting rules, the AI will shape its output accordingly. You can also use markdown in the instructions if your UI expects certain formatting (e.g., "Use bullet points for lists, bold key terms, and use level-2 headings for sections if the answer is long.").

- **Consistent Markdown or HTML Styling:** If your front-end displays answers in a certain way (like in an "answer card"), format the output to fit. For instance, you might have the AI output: **Answer**: *(the answer text)* followed by **Sources**: *(a list of sources)*. Ensure it always uses the same headings or list style for sources. This regularity makes it feel like a scripted, reliable system answer. You might program the model to produce something like:

CoreNeural Answer:
 Detailed answer here in a couple of paragraphs or points.

 Supporting Docs:
 - Doc1 title (source link 1)
 - Doc2 title (source link 2)

And so on, which your UI can render with your branding. The key is to **wrap the model's content in your branded container**. Often, it's safest to do this wrapping in your application code rather than relying on the model to include phrases like "CoreNeural Answer" in text (to avoid the model forgetting or messing it up). Your front-end can simply prepend a label or use a template to display the answer under a CoreNeural banner or card.

- **Use Retrieval-Augmented Generation (RAG) Formatting:** Since you plan to implement a RAG layer, take advantage of that to standardize answers. For example, provide the retrieved document snippets to ChatGPT with clear instructions to **cite them or quote them** in the answer. You can enforce a format where any statement of fact is backed by a citation (perhaps as a reference number or markdown link). Then in post-processing, you can convert those references into footnotes or tooltips in the UI. Because you control how sources are shown, it appears as a proprietary feature of CoreNeural. The user sees, for instance, *"According to Acme Corp's policy document, all data is encrypted at rest* [1] *."* – which looks professional and *product-specific* rather than a generic ChatGPT answer. (Keep the citation style consistent every time.)

- **Post-Process for Formatting Consistency:** Even with good prompt instructions, the model might occasionally format something incorrectly (e.g., it might produce a numbered list when you wanted bullets, or forget a heading). Implement a lightweight **formatting validator** in your code: after the model responds, check if it includes the expected sections or markers. If not, you can either automatically adjust it or ask the model again with a nudge (though automatic is faster). For example, if your standard is "Answer / Sources" and the model gives a single block of text, your code could split it or prepend the missing header. Similarly, strip out any Markdown elements you don't want (like if the model includes tables or images markdown unexpectedly). By normalizing the format post-response, you ensure the output *always* adheres to your UI's look and feel.

- **Embed Tone/Format Rules in Instructions:** You can include formatting rules as part of the system message so the model does a lot of the work for you. For instance: *"Always format answers in bullet points if listing multiple items, and use a neutral, informative tone."* In fact, experts advise treating these like macros – e.g. *"Always format in bullet points"* or *"highlight next actions first"* as standing instructions [4] . This way the model's default behavior is already aligned, reducing how much you need to clean up. One example from industry: *"Every time I ask for a summary, format it with a headline, 3 bullets, and a key action."* [5] . You can adapt this idea for CoreNeural: e.g., *"For any answer longer than a paragraph, use short paragraphs with clear subheadings for each theme, so it's easy to read for executives."* Such granular rules baked into the prompt will yield outputs that require minimal tweaking.

4. Implementation Tips (FastAPI Pipeline)

Finally, putting it all together in your Python/FastAPI backend:

- **API Layer Control:** Have your FastAPI endpoint handle the full cycle:
- **Receive user query** (with an auth token/session id if needed for enterprise).
- **Pre-process & sanitize** the query (as discussed: remove injections, check moderation).
- **Retrieve relevant data** (your RAG step: e.g., vector search your docs and pull top snippets).
- **Construct the prompt** with your system message, user query, and retrieved context. Make sure the system message includes both the *tone/format instructions* and *don't reveal identity instructions*. The retrieved context can be appended as additional messages or inserted into the user prompt (with clear delimiters so the model knows what to cite).
- **Call the OpenAI API** (ChatCompletion) with the composed messages.
- **Post-process the response**:
    - Strip any disallowed content or references (sanitization).
    - Ensure it matches the format (add any wrappers or fix minor format issues).
    - Possibly run a final moderation check on the output (to be extra safe for enterprise compliance).

- **Return the answer** to the user, packaged as your product's response (JSON with your own fields, or rendered HTML/Markdown that fits your frontend).

- **Testing and QA:** Do thorough testing with various queries. For sanitization, test edge cases like users asking *"What AI is this?"* or giving prompts trying to break it. For tone/format, test both straightforward queries and complex ones to see if the answer consistently follows your guidelines. Have some beta users or team members review the answers – do they feel like *CoreNeural*? If anything feels off (too verbose, too informal, a stray "As an AI..."), refine your system prompt or post-processing until it's spot-on.

- **Continuous Improvement:** As users interact, you might discover new phrases to sanitize or slight tone tweaks needed. Maintain a list or config for these (e.g., a list of blacklisted words/phrases to remove, which you can update over time without redeploying the model). Also, keep an eye on OpenAI model updates – new versions might behave differently in tone or might reduce the incidence of unwanted phrases. Adjust your prompts when upgrading models to account for those changes.

By combining **robust sanitization** (input filtering, output cleansing) with **careful tone and format control** (prompt engineering plus post-formatting), your CoreNeural product's responses will appear fully proprietary. The answers will consistently use your company's voice, follow your stylistic rules, and *never* mention ChatGPT or OpenAI. To the end-users and enterprise clients, it will simply feel like they're interacting with CoreNeural's own intelligent, secure assistant – exactly as intended.

Sources:

- Snyk Learn – *Prompt Injection Mitigation* (recommends input sanitization, structural role separation, and pattern filtering to remove injection attempts) [1].
- *The AI Enterprise* (Mark Hinkle) – Advice on customizing ChatGPT's style: keep responses concise, structured, and *"avoid … AI-sounding phrasing"* for a professional tone [3] [4].

---

[1] [2] What is prompt injection? | Tutorial and examples | Snyk Learn
https://learn.snyk.io/lesson/prompt-injection/

[3] [4] [5] Custom Instructions For Better ChatGPT Results
https://www.theaienterprise.io/p/custom-instructions-chatgpt